Московский Авиационный Институт

(Национальный Исследовательский Университет)

Институт №8 “Компьютерные науки и прикладная математика”

Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №3 по курсу

«Операционные системы»

Группа: М8О-210БВ-25

Студент: Хабиров Р.И.

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 04.11.26

Москва, 2026

Постановка задачи

Вариант 7.

Родительский процесс создает дочерний процесс. Первой строчкой пользователь в консоль родительского процесса вводит имя файла, которое будет использовано для открытия файла с таким именем на чтение. Стандартный поток ввода дочернего процесса переопределяется открытым файлом. Дочерний процесс читает команды из стандартного потока ввода. Дочерний процесс записывает данные в Shared Memory. Родительский процесс читает данные из Shared Memory и прочитанное выводит в свой стандартный поток вывода. Родительский и дочерний процесс должны быть представлены разными программами.

В файле записаны команды вида: «число число число<endline>». Дочерний процесс считает их сумму и передает её родительскому процессу, тот в свою очередь выводит её в стандартный поток вывода. Числа имеют тип float. Количество чисел может быть произвольным.

Общий метод и алгоритм решения

Использованные системные вызовы:

- int shm_open(const char *name, int oflag, mode_t mode); - создание или открытие именованного объекта разделяемой памяти.
- int shm_unlink(const char *name); - удаление имени объекта разделяемой памяти.
- int ftruncate(int fd, off_t length); - установка размера объекта разделяемой памяти.
- void *mmap(void *addr, size_t length, int prot, int flags, int fd, off_t offset); - отображение объекта памяти в виртуальное адресное пространство процесса.
- int munmap(void *addr, size_t length); - удаление отображения памяти из адресного пространства процесса.
- sem_t *sem_open(const char *name, int oflag, ...); - создание или открытие именованного семафора (при создании дополнительно указываются права доступа и начальное значение).
- int sem_wait(sem_t *sem); - ожидание семафора и уменьшение его счётчика на единицу.
- int sem_post(sem_t *sem); - увеличение счётчика семафора и возможное разблокирование ожидающего процесса.
- int sem_unlink(const char *name); - удаление имени семафора.
- int sem_close(sem_t *sem); - закрытие именованного семафора в текущем процессе.

В основе программы лежит механизм межпроцессного взаимодействия с использованием именованной разделяемой памяти (POSIX Shared Memory) и её отображения в виртуальное адресное пространство процессов (Memory Mapping). Для синхронизации чтения и записи данных используются именованные POSIX-семафоры.
Программа состоит из двух отдельных программ: родительского и дочернего процессов. Родительский процесс получает имя файла от пользователя, открывает его и создаёт дочерний процесс. Дочерний процесс считывает из файла вещественные числа типа float, вычисляет их сумму для каждой строки и передаёт результаты родителю через разделяемую память. Родительский процесс выводит полученные суммы в стандартный поток вывода.
Для передачи данных используется общая структура SharedData, содержащая два поля:

- sum - сумма чисел текущей строки, имеющая тип float;
- status - целочисленный статус сообщения (1 - передача результата, 2 - завершение обработки файла, любое другое значение – сообщение о некорректном поведении, завершение с ошибкой).

Синхронизация осуществляется двумя семафорами:

- empty - имеет начальное значение 1 и разрешает дочернему процессу запись в общую память;
- full - имеет начальное значение 0 и сигнализирует родительскому процессу о наличии готовых данных.

Алгоритм решения:

1. Родительский процесс считывает из стандартного потока ввода имя файла. Затем формирует уникальные имена объекта разделяемой памяти и двух семафоров с использованием идентификатора процесса (PID).
2. С помощью функции shm_open родитель создаёт именованный объект разделяемой памяти. Функция ftruncate устанавливает его размер, равный размеру структуры SharedData. Затем mmap отображает объект в виртуальное адресное пространство родительского процесса с флагом MAP_SHARED.
3. Родительский процесс создаёт два именованных семафора с помощью sem_open, задавая им начальные значения 1 и 0 соответственно. После этого открывает указанный пользователем файл на чтение.
4. С помощью fork создаётся дочерний процесс. В дочерней ветке стандартный поток ввода перенаправляется на открытый файл с помощью dup2. Затем функция execve запускает отдельную программу child, передавая ей через аргументы командной строки имена разделяемой памяти и семафоров.
5. Дочерняя программа подключается к существующему объекту разделяемой памяти с помощью shm_open и mmap, а также открывает созданные родителем семафоры через sem_open. Далее последовательно считывает символы из стандартного потока ввода, преобразует их в числа типа float и вычисляет сумму чисел каждой строки.
6. После вычисления очередной суммы дочерний процесс вызывает sem_wait(empty), ожидая освобождения общей памяти. Затем записывает сумму в поле sum, устанавливает status = 1 и вызывает sem_post(full), уведомляя родительский процесс о готовности результата.
7. Родительский процесс ожидает данные с помощью sem_wait(full). После получения сообщения со статусом 1 он считывает сумму из разделяемой памяти, преобразует её в строку и выводит в стандартный поток вывода. Затем вызывает sem_post(empty), разрешая дочернему процессу записать следующий результат.
8. После обработки всех строк дочерний процесс передаёт сообщение со статусом 2, сигнализирующее о завершении работы. Родительский процесс получает это сообщение и выходит из цикла чтения результатов.
9. Дочерний процесс освобождает своё отображение памяти с помощью munmap и закрывает семафоры через sem_close. Родительский процесс ожидает завершения дочернего с помощью waitpid, после чего освобождает собственное отображение памяти, закрывает семафоры и удаляет имена созданных объектов с помощью shm_unlink и sem_unlink.

Код программы

Заголовочный файл shared.h

```
#ifndef SHARED_H

#define SHARED_H
```

```
typedef struct {
    float sum;
    int status;
}SharedData;


#endif
```

Программа child.c

```
#include <unistd.h>
#include <fcntl.h>
#include <sys/mman.h>
#include <sys/stat.h>
#include <semaphore.h>
#include "shared.h"

int str_len(const char *str){
    int len = 0;
    while (str[len] != '\0'){
        len ++;
    }
    return len;
}

float parse_float(const char* str){
    float res = 0.0;
    float frac = 1.0;
    int i = 0;
    int is_frac = 0;
    int sign = 1;
    if (str[0] == '-'){
```

```
        sign = -1;
        i ++;
    }
    for (; str[i] != '\0'; i ++){
        if (str[i] == '.'){
            is_frac = 1;
            continue;
        }
        if (str[i] >= '0' && str[i] <= '9'){
            if (!is_frac){
                res = res * 10.0 + (str[i] - '0');
                continue;
            }
            else {
                frac = frac / 10;
                res = res + (str[i] - '0') * frac;
            }
        }
    }
    return res * sign;
}

int main(int argc, char *argv[]){
    SharedData *data;
    sem_t *empty;
    sem_t *full;
    if (argc != 4){
        const char *msg = "Некорректное использование\n";
        write(2, msg, str_len(msg));
        return 1;
```

```
    }
    int shm_fd = shm_open(argv[1], O_RDWR, 0);
    if (shm_fd == -1){
        const char *msg = "Ошибка shm_open\n";
        write(2, msg, str_len(msg));
        return 1;
    }

    data = mmap(NULL, sizeof(SharedData), PROT_READ | PROT_WRITE, MAP_SHARED,
shm_fd, 0);
    if (data == MAP_FAILED){
        const char *msg = "Ошибка mmap\n";
        write(2, msg, str_len(msg));
        close(shm_fd);
        return 1;
    }
    close(shm_fd);

    empty = sem_open(argv[2], 0);
    if (empty == SEM_FAILED){
        const char *msg = "Ошибка семафора empty\n";
        write(2, msg, str_len(msg));
        munmap(data, sizeof(SharedData));
        return 1;
    }

    full = sem_open(argv[3], 0);
    if (full == SEM_FAILED){
        const char *msg = "Ошибка семафора full\n";
        write(2, msg, str_len(msg));
```

```
    munmap(data, sizeof(SharedData));
    sem_close(empty);
    return 1;
}

char num_buff[64], ch;
int num_len = 0, has_num = 0;
float line_sum = 0.0f;
while (read(0, &ch, 1) > 0){
    if ((ch >= '0' && ch <= '9') || ch == '.' || (ch == '-' && num_len == 0)){
        if (num_len < 63){
        num_buff[num_len ++] = ch;
        }
    }
    else{
        if (num_len > 0){
            num_buff[num_len] = '\0';
            line_sum += parse_float(num_buff);
            num_len = 0;
            has_num = 1;
        }
        if (ch == '\n'){
            if (has_num){
                sem_wait(empty);
                data->sum = line_sum;
                data->status = 1;
                sem_post(full);
            }
            line_sum = 0.0f;
            has_num = 0;
```

```
            }
        }
    }
    if (num_len > 0){
        num_buff[num_len] = '\0';
        line_sum += parse_float(num_buff);
        num_len = 0;
        has_num = 1;
    }
    if (has_num){
        sem_wait(empty);
        data->sum = line_sum;
        data->status = 1;
        sem_post(full);
    }

    sem_wait(empty);
    data->status = 2;
    sem_post(full);

    munmap(data, sizeof(SharedData));
    sem_close(full);
    sem_close(empty);
    return 0;
}
```

Программа parent.c

```
#include <unistd.h>
#include <math.h>
#include <fcntl.h>
#include <sys/wait.h>
```

```
#include <sys/types.h>
#include <sys/mman.h>
#include <sys/stat.h>
#include <semaphore.h>
#include "shared.h"
#include <stdbool.h>

int str_len(const char *str){
    int len = 0;
    while (str[len] != '\0'){
        len ++;
    }
    return len;
}

void pid_str(char buff[64], pid_t num) {
    int temp_num = (int) num;
    int len_num = 0;

    if (temp_num == 0){
        buff[0] = '0';
        buff[1] = '\0';
        return;
    }

    while (temp_num > 0) {
        len_num++;
        temp_num /= 10;
    }
```

```
    int i = len_num - 1;
    temp_num = (int) num;
    buff[i+1] = '\0';
    while (temp_num > 0) {
        buff[i--] = (temp_num % 10) + '0';
        temp_num /= 10;
    }

}

void make_shm_name(char *buffer, pid_t pid){
    char buff[64];
    const char *prefix = "/lab3_shm_";
    int len = str_len(prefix);
    int i = 0;

    pid_str(buff, pid);
    while (len != 0){
        buffer[i] = prefix[i];
        i++;
        len--;
    }
    len = str_len(buff);
    int buff_i = 0;
    while (len != 0){
        buffer[i] = buff[buff_i];
        i ++;
        buff_i ++;
        len --;
    }
```

```
    buffer[i] = '\0';
}

void make_empty_name(char *buffer, pid_t pid){
    char buff[64];
    const char *prefix = "/lab3_empty_";
    int len = str_len(prefix);
    int i = 0;

    pid_str(buff, pid);
    while (len != 0){
        buffer[i] = prefix[i];
        i++;
        len--;
    }
    len = str_len(buff);
    int buff_i = 0;
    while (len != 0){
        buffer[i] = buff[buff_i];
        i ++;
        buff_i ++;
        len --;
    }
    buffer[i] = '\0';
}

void make_full_name(char *buffer, pid_t pid){
    char buff[64];
    const char *prefix = "/lab3_full_";
    int len = str_len(prefix);
```

```
    int i = 0;

    pid_str(buff, pid);
    while (len != 0){
        buffer[i] = prefix[i];
        i++;
        len--;
    }
    len = str_len(buff);
    int buff_i = 0;
    while (len != 0){
        buffer[i] = buff[buff_i];
        i ++;
        buff_i ++;
        len --;
    }
    buffer[i] = '\0';
}

int float_str(char buff[64], float num) {
    int i = 0;

    if (num < 0) {
        buff[i++] = '-';
        num = fabs(num);
    }

    long long int res = (long long int)num;
    int int_frac = (int)((num - res) * 100.0f + 0.5f);
```

```
if (int_frac >= 100) {
    res += 1;
    int_frac -= 100;
}

if (res == 0) {
    buff[i++] = '0';
}
else {
    long long int temp_res = res;
    int len_res = 0;
    while (temp_res > 0) {
        len_res++;
        temp_res /= 10;
    }

    int temp_i = i + len_res - 1;
    while (res > 0) {
        buff[temp_i--] = (res % 10) + '0';
        res /= 10;
    }
    i += len_res;
}

buff[i++] = '.';

buff[i++] = (int_frac / 10) + '0';
buff[i++] = (int_frac % 10) + '0';

buff[i++] = '\n';
```

```
    buff[i] = '\0';

    return i;
}

int main(){
    char filename[256];
    char shm_name[64];
    char empty_name[64];
    char full_name[64];
    SharedData *data;
    sem_t *empty;
    sem_t *full;
    int result_code = 0;
    const char *msg = "Введите имя файла: ";
    write(1, msg, str_len(msg));
    ssize_t bytes_read = read(0, filename, sizeof(filename) - 1);

    if (bytes_read <= 0){
        return 1;
    }

    filename[bytes_read] = '\0';

    for (int i = 0; i < bytes_read; i ++){
        if (filename[i] == '\n'){
            filename[i] = '\0';
            break;
        }
    }
```

```
    make_shm_name(shm_name, getpid());
    int shm_fd = shm_open(shm_name, O_CREAT|O_EXCL|O_RDWR, 0600);
    if (shm_fd == -1){
        const char *msg = "Ошибка создания Shared Memory\n";
        write(2, msg, str_len(msg));
        return 1;
    }
    if (ftruncate(shm_fd, sizeof(SharedData)) == -1){
        const char *msg = "Ошибка работы ftruncate\n";
        write(2, msg, str_len(msg));
        close(shm_fd);
        shm_unlink(shm_name);
        return 1;
    }

    data = mmap(NULL, sizeof(SharedData), PROT_READ | PROT_WRITE, MAP_SHARED,
shm_fd, 0);

    if (data == MAP_FAILED){
        const char *msg = "Ошибка в работе mmap\n";
        write(2, msg, str_len(msg));
        close(shm_fd);
        shm_unlink(shm_name);
        return 1;
    }

    close(shm_fd);

    make_empty_name(empty_name, getpid());
```

```
make_full_name(full_name, getpid());

empty = sem_open(empty_name, O_CREAT | O_EXCL, 0600, 1);
if (empty == SEM_FAILED){
        const char *msg = "Ошибка в создании семафора empty\n";
        write(2, msg, str_len(msg));
        shm_unlink(shm_name);
        munmap(data, sizeof(SharedData));
        return 1;
    }

full = sem_open(full_name, O_CREAT | O_EXCL, 0600, 0);

if (full == SEM_FAILED){
    const char *msg = "Ошибка в создании семафора full\n";
    write(2, msg, str_len(msg));
    shm_unlink(shm_name);
    munmap(data, sizeof(SharedData));
    sem_close(empty);
    sem_unlink(empty_name);
    return 1;
}

int file_fd = open(filename, O_RDONLY);
if (file_fd == -1){
    const char *msg = "Не удалось открыть файл\n";
    write(2, msg, str_len(msg));
    shm_unlink(shm_name);
    munmap(data, sizeof(SharedData));
    sem_unlink(empty_name);
```

```
    sem_close(empty);
    sem_unlink(full_name);
    sem_close(full);
    return 1;
}

pid_t pid = fork();

if (pid == -1){
    const char *msg = "Ошибка fork\n";
    write(2, msg, str_len(msg));
    shm_unlink(shm_name);
    munmap(data, sizeof(SharedData));
    sem_unlink(empty_name);
    sem_close(empty);
    sem_unlink(full_name);
    sem_close(full);
    close(file_fd);
    return 1;
}

else if(pid == 0){
    char *child_argv[] = {"./child", shm_name, empty_name, full_name, NULL};
    char *child_envp[] = {NULL};

    if (dup2(file_fd, 0) == -1){
        const char *msg = "Ошибка dup2\n";
        write(2, msg, str_len(msg));
        _exit(1);
    }
```

```
    close(file_fd);

    execve("./child", child_argv, child_envp);

    const char *msg = "Не удалось запустить ./child\n";
    write(2, msg, str_len(msg));
    _exit(1);
}

else{
    close(file_fd);
    char buff[64];

    const char *msg = "\nРезультаты из дочернего процесса\n";
    write(1, msg, str_len(msg));

    while (true){
        sem_wait(full);
        if (data->status == 1){
            int len = float_str(buff, data->sum);
            write(1, buff, len);
            sem_post(empty);
        }
        else if(data->status == 2){
            sem_post(empty);
            break;
        }
        else{
            const char *msg = "Произошла ошибка\n";
            write(2, msg, str_len(msg));
```

```
            sem_post(empty);
            result_code = 1;
            break;
        }
    }

    int status;
    waitpid(pid, &status, 0);
    shm_unlink(shm_name);
    munmap(data, sizeof(SharedData));
    sem_unlink(empty_name);
    sem_close(empty);
    sem_unlink(full_name);
    sem_close(full);
  }

  return result_code;
}
```

Протокол работы программы

```
flosy@Flosy:~$ cd 2_course/OS-labs/lab_3
flosy@Flosy:~/2_course/OS-labs/lab_3$ gcc parent.c -o parent -lm
flosy@Flosy:~/2_course/OS-labs/lab_3$ gcc child.c -o child
flosy@Flosy:~/2_course/OS-labs/lab_3$ cat f1
1.5 2.5 3.0
4.2 1.8
10.5 -2.5 1.0
flosy@Flosy:~/2_course/OS-labs/lab_3$ ./parent
Введите имя файла: f1

Результаты из дочернего процесса
```

```
7.00
6.00
9.00
flosy@Flosy:~/2_course/OS-labs/lab_3$ cat f2
1.5 2.5
3.0 4.0
flosy@Flosy:~/2_course/OS-labs/lab_3$ ./parent
Введите имя файла: f2

Результаты из дочернего процесса
4.00
7.00
flosy@Flosy:~/2_course/OS-labs/lab_3$ cat f3
1.1 2.2 3.3
flosy@Flosy:~/2_course/OS-labs/lab_3$ ./parent
Введите имя файла: f3

Результаты из дочернего процесса
6.60
flosy@Flosy:~/2_course/OS-labs/lab_3$ cat f4
flosy@Flosy:~/2_course/OS-labs/lab_3$ ./parent
Введите имя файла: f4

Результаты из дочернего процесса
flosy@Flosy:~/2_course/OS-labs/lab_3$ cat f5
-5.5 2.0 -1.5
-10.0 3.0
flosy@Flosy:~/2_course/OS-labs/lab_3$ ./parent
Введите имя файла: f5
```

Результаты из дочернего процесса

-5.00

-7.00

flosy@Flosy:~/2_course/OS-labs/lab_3$ cat f6

1.0 2.0

3.0 4.0

flosy@Flosy:~/2_course/OS-labs/lab_3$ ./parent

Введите имя файла: f6

Результаты из дочернего процесса

3.00

7.00

Вывод

В ходе данной лабораторной работы я приобрёл практические навыки организации межпроцессного взаимодействия в операционной системе Linux с использованием разделяемой памяти (Shared Memory) и механизма отображения памяти (Memory Mapping). Были изучены способы создания и использования именованных объектов разделяемой памяти, а также синхронизация процессов с помощью POSIX-семафоров. Программа была успешно реализована и протестирована.